

ProjectLens AI 코어타임 인포그래픽

목표는 ProjectLens 서비스 소개가 아니라, 과제의 AI 필수 요구인 RAG, MCP, Agent를 정확히 이해하고 구현 판단 기준을 잡는 것이다.

RAG

- 내 데이터 검색
- 학습이 아니라 retrieval
- 검색 결과를 context로 주입

MCP

- 외부 시스템 연결
- Host/Client/Server 구조
- JSON-RPC 기반 tool 호출

Agent

- 상태와 루프 관리
- Think -> Act -> Observe
- 도구 선택과 최종 output

발표에서 말할 것

- 세 기술의 본질: 검색, 연결, 루프
- 과제가 왜 이 세 가지를 요구하는지
- Agent가 RAG와 MCP 결과를 보고 다시 판단하는 구조
- 비용과 크레딧 폭주를 막는 운영 감각

구현할 때 참고할 것

- embedding, vector DB, threshold, source metadata
- MCP Server, tool schema, JSON-RPC, 권한 관리
- function calling, state, max turns, error handling
- 기존 backend/frontend 코드와 무엇이 비슷하고 다른지

한 줄로 외우기

RAG는 검색, MCP는 연결, Agent는 루프다. 모델은 판단하고, backend는 실행과 권한을 통제한다.

게시판 기본 기능 위에 AI 필수 요구를 얹는다

게시판 기본 요구

- CRUD
- 인증
- 댓글
- 검색/태그
- 페이지네이션
- 정렬/상태 UI

AI 요구는 게시판과 별개로 평가되는 구현 책임이다.

필수 기술	과제 설명의 의미	구현에서 보여줘야 하는 것
RAG	개인 또는 사내 데이터와 LLM을 연결	embedding/vector DB에 저장하고 질문에 맞게 검색해 context로 넣는다.
MCP	LLM이 외부 시스템을 호출	MCP Server가 tool을 제공하고 Host/Client가 JSON-RPC 흐름으로 호출한다.
Agent	스스로 도구 선택과 실행 루프 관리	function calling, state, observation, max turns, 예외처리, structured output을 보여준다.

RAG 최소선

- 데이터 소스
- embedding 모델
- vector DB
- similarity search
- 검색 결과의 출처

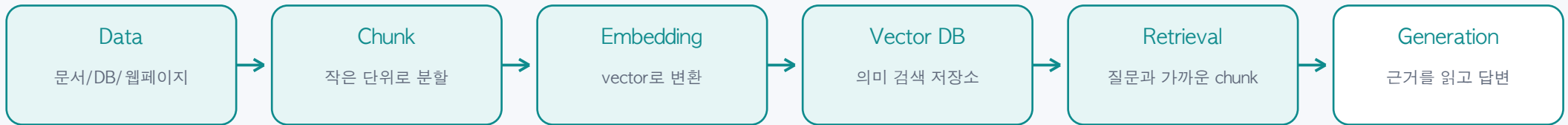
MCP 최소선

- MCP Server
- tool schema
- JSON-RPC 이해
- 외부 서비스 1개 이상
- API key/ 권한 관리

Agent 최소선

- function calling
- state/memory
- tool observation
- 무한 루프 방지
- 예외처리

학습이 아니라 검색해서 읽게 하는 구조



오해 -> 정확한 설명

- 우리 데이터를 모델에 학습시킨 것? 아니다. 검색 결과를 현재 context로 넣은 것이다.
- 문서 전체를 프롬프트에 붙이면 RAG? 아니다. 핵심은 retrieval 계층이다.
- RAG면 정답 보장? 아니다. 검색 품질과 출처 검증이 같이 필요하다.

결정할 것

좋은 기준

데이터 소스	LLM이 원래 모르지만 답변에 필요한 데이터
chunking	맥락이 끊기지 않도록 token 크기와 overlap을 조정
embedding	저장 데이터와 질문을 같은 모델로 vector화
vector DB	Pinecone, FAISS, Chroma, pgvector 등 운영 맥락에 맞게 선택
metadata	source id, 제목, URL, 작성일, score를 남겨 검증 가능하게 함
fallback	threshold 미달이면 추측하지 않고 근거 부족 상태로 처리

이미지 자리

RAG indexing -> retrieval -> generation 흐름
출처: Notion [JUNGLE] AI로 진화하기 / RAG
주의: signed URL은 PDF에 직접 삽입하지 않는다.

● 기존 LIKE 검색

```
SELECT * FROM posts
WHERE title LIKE '%refund%';
```

● RAG semantic search

```
distance = Embedding.embedding.cosine_distance(query_vector)
stmt = stmt.order_by(distance).limit(limit)
```

발표 문장

RAG는 AI가 데이터를 외운 것이 아니라, 필요한 순간에 관련 데이터를 찾아 읽는 구조입니다.

외부 API를 AI가 쓰기 쉬운 tool 계층으로 감싼다



USB-C 비유

이미지 자리: MCP USB-C 비유
출처: Notion MCP 페이지
의도: AI 앱과 외부 도구의 공통 연결 포트라는 감각을 보여준다.

+ 2

결정할 것	구현 기준
MCP Server	어떤 기능을 tool로 제공할지 정한다.
tool schema	LLM이 안정적으로 arguments를 만들 수 있게 입력/출력 구조를 명확히 한다.
외부 서비스	GitHub, Notion, Slack, Google 등 실제 서비스 1개 이상을 연결한다.
JSON-RPC	Client/Server 사이의 tools/list, tools/call 흐름을 이해한다.
권한/키	secret은 서버 환경에 두고 tool result에는 노출하지 않는다.
보안	allowlist, timeout, body limit, URL 검증을 둔다.

익숙한 backend API wrapper

```
@router.get("/github/readme")
async def readme(github_url: str):
    return await fetch_github_readme(github_url)
```

MCP tool

```
@mcp.tool()
async def fetch_github_readme(github_url: str) -> dict[str, Any]:
    return await fetch_github_readme_tool(github_url)
```

단일 LLM 호출이 아니라 상태와 observation을 가진 루프



Model

- 목표와 context를 읽고 판단
- tool_name + arguments 제안

핵심

Tools

- 검색, 외부 호출, 계산, 저장
- 입출력 schema가 명확해야 함

변수

Orchestration

- 상태, 순서, retry, 종료 조건
- backend가 실행 권한 통제

결정할 것

좋은 기준

function calling	모델은 호출할 함수와 arguments를 만들고 앱이 실제 함수를 실행
state	running/completed/failed, tool 결과, 최종 report를 저장
loop 제한	max turns, timeout, retry limit으로 무한 루프 방지
예외 처리	failed/need_more_info/refused/loading 같은 상태 구분
output	UI와 DB가 읽을 수 있는 structured output 사용

● Agent 구조 예시

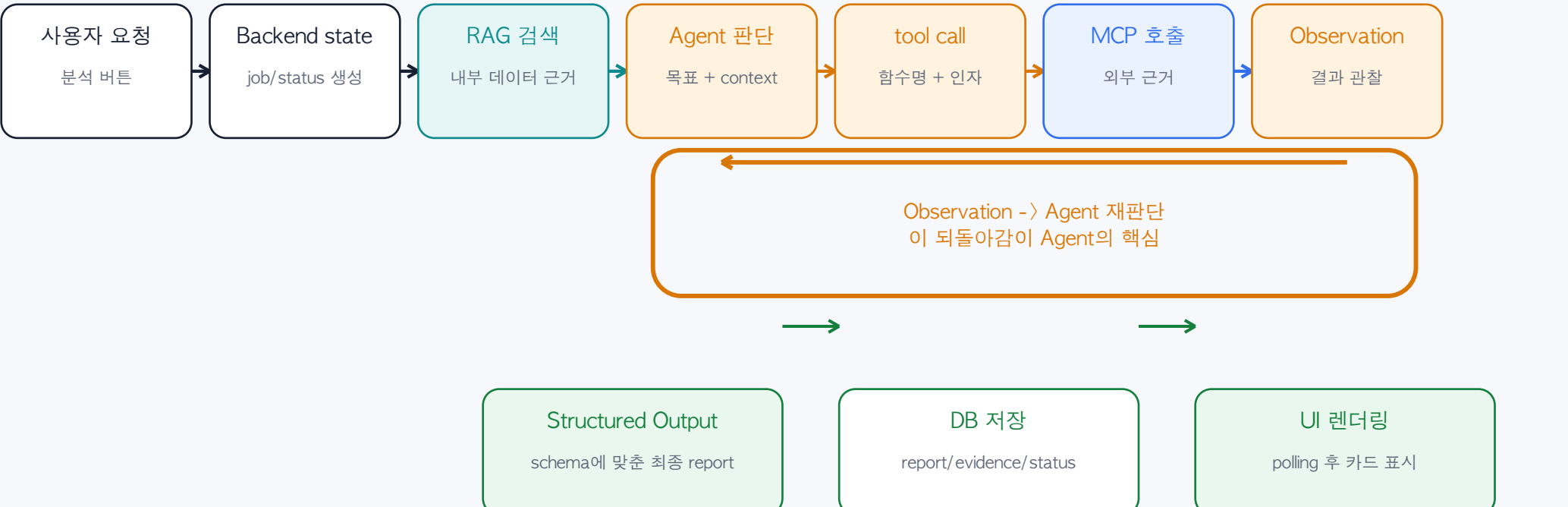
```

return Agent(
    instructions=PROJECT_ANALYSIS_INSTRUCTIONS,
    tools=analysis_tools,
    model=settings.agent_model,
    output_type=ProjectAnalysisReport,
)
  
```

발표 문장

Agent는 프롬프트가 긴 LLM 호출이 아니라, 생각하고 도구를 쓰고 결과를 관찰한 뒤 다시 판단하는 실행 구조입니다.

Agent 루프 안에서 RAG와 MCP가 도구처럼 쓰인다



일반 흐름		ProjectLens 예시 라벨	
사용자 요청		게시글 상세의 AI 분석 버튼	
상태 생성		analysis job endpoint와 posts.analysis_status	
RAG		embedding 기반 유사 프로젝트 검색	
tool call/MCP		function_tool wrapper -> local/private MCP Server	
Observation		사이트/GitHub 근거 결과	
최종 output		Pydantic Structured Outputs -> ai_reports, mcp_evidences	
UI		분석 상태 polling 후 카드 UI 렌더링	

Sources: local draft section 4, repo implementation labels

발표 중 길게 읽지 말고, 만들 때 확인하는 결정표

영역	결정	질문	좋은 기준
RAG	데이터 소스	무엇을 검색할 것인가?	LLM이 원래 모르지만 답변에 꼭 필요한 데이터만 넣는다.
RAG	embedding/vector DB	어떻게 의미 검색할 것인가?	같은 embedding 모델로 저장/질문 vector를 만들고 pgvector/FAISS/Chroma/Pinecone 중 운영에 맞게 선택한다.
RAG	metadata/fallback	검색 결과를 믿을 수 있는가?	source, score, 작성일을 저장하고 threshold 미달은 근거 부족으로 처리한다.
MCP	MCP Server	어떤 외부 기능을 tool로 제공할 것인가?	범용 call_api보다 fetch_github_readme처럼 목적이 분명한 tool을 둔다.
MCP	JSON-RPC/tool schema	LLM이 안정적으로 호출할 수 있는가?	tools/list와 tools/call 흐름, 입력 schema, 출력 형태를 설명 가능하게 만든다.
MCP	권한/보안	API key와 외부 입력을 어떻게 제한할 것인가?	secret은 서버에 두고 allowlist, timeout, body limit, URL 검증을 적용한다.
Agent	function calling	모델과 backend의 책임을 나눴는가?	모델은 tool_name + arguments, backend는 실행과 검증을 맡는다.
Agent	state/loop 제한	무한 루프와 새로고침을 견딜 수 있는가?	running/completed/failed, max turns, timeout, retry limit을 둔다.
Agent	structured output	UI가 안정적으로 렌더링할 수 있는가?	자유 텍스트 대신 schema 기반 결과를 저장한다.

읽는 방법

이 표는 발표용 문장이 아니라 구현 체크리스트다. 발표에서는 'RAG는 검색 구조, MCP는 외부 tool 연결, Agent는 그 둘을 쓰는 루프'만 먼저 잡아준다.

RAG 개선은 검색 품질을 끌어올리는 일이다

문제 신호

- 검색 결과가 엉뚱하다.
- 관련 없는 문서가 context에 섞인다.
- 검색 결과가 없는데 그럴듯한 답을 한다.
- 출처가 없어 검증이 어렵다.

조정할 레버

- chunk 크기와 overlap
- embedding 입력 텍스트
- top-k와 similarity threshold
- metadata와 source 표시

고급 개선

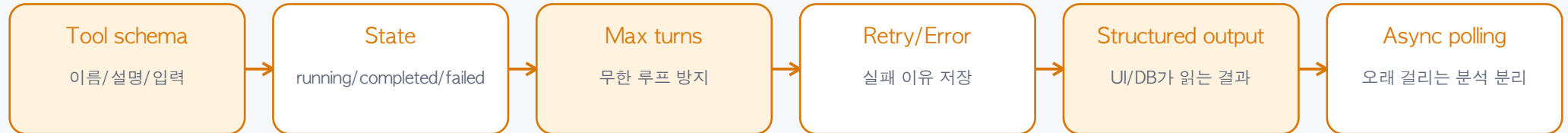
- semantic score 외 ranking signal 추가
- 태그, 최신성, 유형 기반 weighted RAG
- 빈 결과를 정상 상태로 처리

문제	개선 방향
검색 결과가 엉뚱함	chunk/metadata/embedding 입력 텍스트를 조정
낮은 관련도 결과가 섞임	threshold와 top-k 조정
근거가 빈약함	source title, URL, 작성일, score를 함께 표시
데이터가 쌓였는데 정렬이 단순함	semantic score 외 태그, 최신성, 유형 signal을 섞음
결과가 없는데 답함	근거 부족 상태로 처리하고 추측을 막음

발표 문장

RAG 품질은 모델이 얼마나 똑똑한가보다 무엇을 검색해 넣었는가에 크게 좌우됩니다. 그래서 검색 결과의 관련도, 출처, 빈 결과 처리가 핵심입니다.

Agent 개선은 루프를 안정적으로 끝내게 만드는 일이다



문제	개선 방향
tool을 잘못 고름	tool name, description, input schema를 구체화
같은 tool을 반복	max turns, retry limit, 중복 호출 방지
실패 이유를 모름	tool error와 status를 저장
응답이 UI에 맞지 않음	structured output schema 강제
분석이 오래 걸림	async job + frontend polling으로 분리
근거와 결론이 섞임	observation은 evidence, 최종 output은 schema로 분리

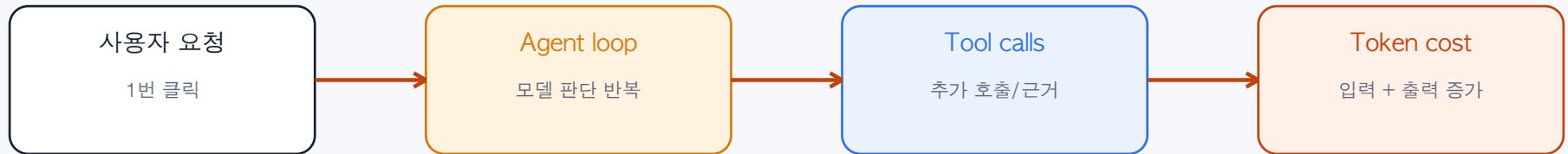
핵심

Agent는 실패와 지연까지 상태로 다뤄야 서비스가 된다. 성공 경로만 있으면 데모는 되지만 운영 가능한 기능이라고 설명하기 어렵다.

ProjectLens 예시

async polling과 structured output retry는 Agent 루프 안정화의 예시다. 발표에서는 예시로만 짧게 언급한다.

Agent 루프는 호출 수와 token 비용을 빠르게 키울 수 있다



Rate limits

- RPM: 분당 요청 수
- TPM: 분당 token 수
- 실패/retry도 제한에 영향을 줄 수 있음

Pricing

- API는 token 기준 과금
- 입력 token과 출력 token 모두 비용 요인
- 가격은 공식 pricing에서 확인

Usage dashboard

- 사용량을 직접 확인
- monthly budget과 알림 threshold 설정
- 한도 반영 지연 가능성 고려

Prepaid credits

- auto recharge 상태 확인
- monthly recharge limit 설정
- 만료/환불 불가 조건 확인

발표에서 꼭 말할 것

ChatGPT 구독과 OpenAI API 과금은 별도입니다. 실습 전에는 usage dashboard, monthly budget, auto recharge 상태를 확인해야 합니다.

실습 권장

- 작은 prepaid credit으로 시작
- Agent max turns를 낮게 설정
- 데모 전 remaining credits 확인
- 대량 embedding은 batch와 rate limit 확인

기존 backend/frontend 감각으로 AI 코드를 해석하기

● RAG: LIKE 검색의 확장

```
SELECT * FROM posts WHERE title LIKE '%refund%';

distance = Embedding.embedding.cosine_distance(query_vector)
stmt = stmt.order_by(distance).limit(limit)
```

● MCP: API wrapper의 AI-friendly 버전

```
@router.get("/github/readme")
async def readme(github_url: str): ...

@mcp.tool()
async def fetch_github_readme(github_url: str) -> dict: ...
```

● Agent: service orchestration의 AI 버전

```
async def run_analysis(post_id: int):
    post = await load_post(post_id)
    evidence = await fetch_evidence(post)
    report = build_report(post, evidence)

Agent(instructions=..., tools=..., output_type=...)
```

해석

둘 다 DB에서 관련 데이터를 찾는다. 차이는 LIKE가 문자열 일치 검색이고, RAG는 embedding vector의 의미적 거리 검색이라는 점이다.

해석

둘 다 외부 서비스를 호출한다. 다른 점은 MCP tool이 Agent/Host가 schema를 보고 호출할 수 있는 표준 tool 표면을 제공한다는 점이다.

해석

일반 service는 개발자가 순서를 고정한다. Agent는 모델이 tool 사용 여부와 arguments를 선택하고, backend가 실행과 권한을 통제한다.

Frontend 연결

기존 fetch 후 렌더링과 비슷하지만, AI 분석은 오래 걸릴 수 있어 start job -> polling status -> completed/failed 카드 렌더링으로 확장된다.

Sources: local draft section 40, repo code reading comparison

외부 결과는 지시문이 아니라 근거 데이터다

보안 원칙

- 외부 사이트, README, MCP 결과는 instruction이 아니라 evidence로만 취급한다.
- API key와 secret은 tool output, 로그, AI context에 넣지 않는다.
- URL fetch는 SSRF 가드, timeout, body limit, redirect 재검증이 필요하다.
- AI 출력은 structured output으로 검증 가능한 형태로 저장한다.

외울 문장

- RAG는 학습이 아니라 검색이다.
- MCP는 API를 AI가 쓰기 쉽게 감싸는 표준 연결 방식이다.
- Agent는 단일 호출이 아니라 Think -> Act -> Observe 루프다.
- 모델은 판단하고 backend는 실행과 권한을 통제한다.

질문	짧은 답
RAG를 쓰면 학습한 건가요?	아니다. 모델 파라미터를 바꾸지 않고 검색 결과를 현재 context로 넣는다.
SQL 검색도 RAG인가요?	retrieval은 가능하지만 과제에서 기대하는 RAG는 보통 embedding 기반 semantic search와 vector DB를 포함한다.
MCP와 API는 뭐가 다른가요?	API는 범용 연결이고 MCP는 AI Agent가 tool로 발견/호출하기 좋은 표준 계층이다.
LangGraph를 꼭 써야 하나요?	핵심은 라이브러리 이름보다 state와 loop가 보이는 agentic 구조다.
왜 structured output이 필요한가요?	UI 카드와 DB 저장을 안정적으로 만들기 위해 필드가 일정해야 한다.
비용은 어디서 커지나요?	Agent가 tool을 여러 번 호출하고 긴 evidence를 읽을 때 token과 요청 수가 늘어난다.

출처: DOCS/ProjectLens_AI파트_코어타임_설명초안.md, AGENTS.md, repo code paths backend/app/ai, backend/app/rag, backend/app/mcp_client, mcp-server, frontend/src/components/analysis. Notion: RAG, MCP, AI agents 페이지의 개념과 이미지 캡션. OpenAI official docs: Rate limits, API Pricing, Prepaid billing, Agents SDK, Function calling, Structured Outputs, Retrieval, MCP and Connectors.